🔔 MEDIATION PRO - ALERT SYSTEM SPECIFICATION

Complete Technical Documentation

Version: 1.0

Last Updated: January 2025

📑 MỤC LỤC

---

1. TỔNG QUAN

1.1 Mục tiêu

Alert System giúp team Marketing/Operations:
- **Phát hiện sớm** các vấn đề về revenue, performance
- **Tự động thông báo** qua nhiều kênh (Telegram, Lark, Slack, Email)
- **Giảm thời gian phản ứng** từ >24h xuống <2h
- **Không bỏ sót** các sự cố quan trọng

1.2 Nguyên tắc hoạt động

```
Data Sources → Rule Evaluation → Deduplication → Priority Routing → Notification
```

1. **Scheduled Check**: Chạy mỗi 15 phút
2. **Rule-based**: 20+ rules được định nghĩa sẵn
3. **Smart Dedup**: Không spam cùng 1 alert

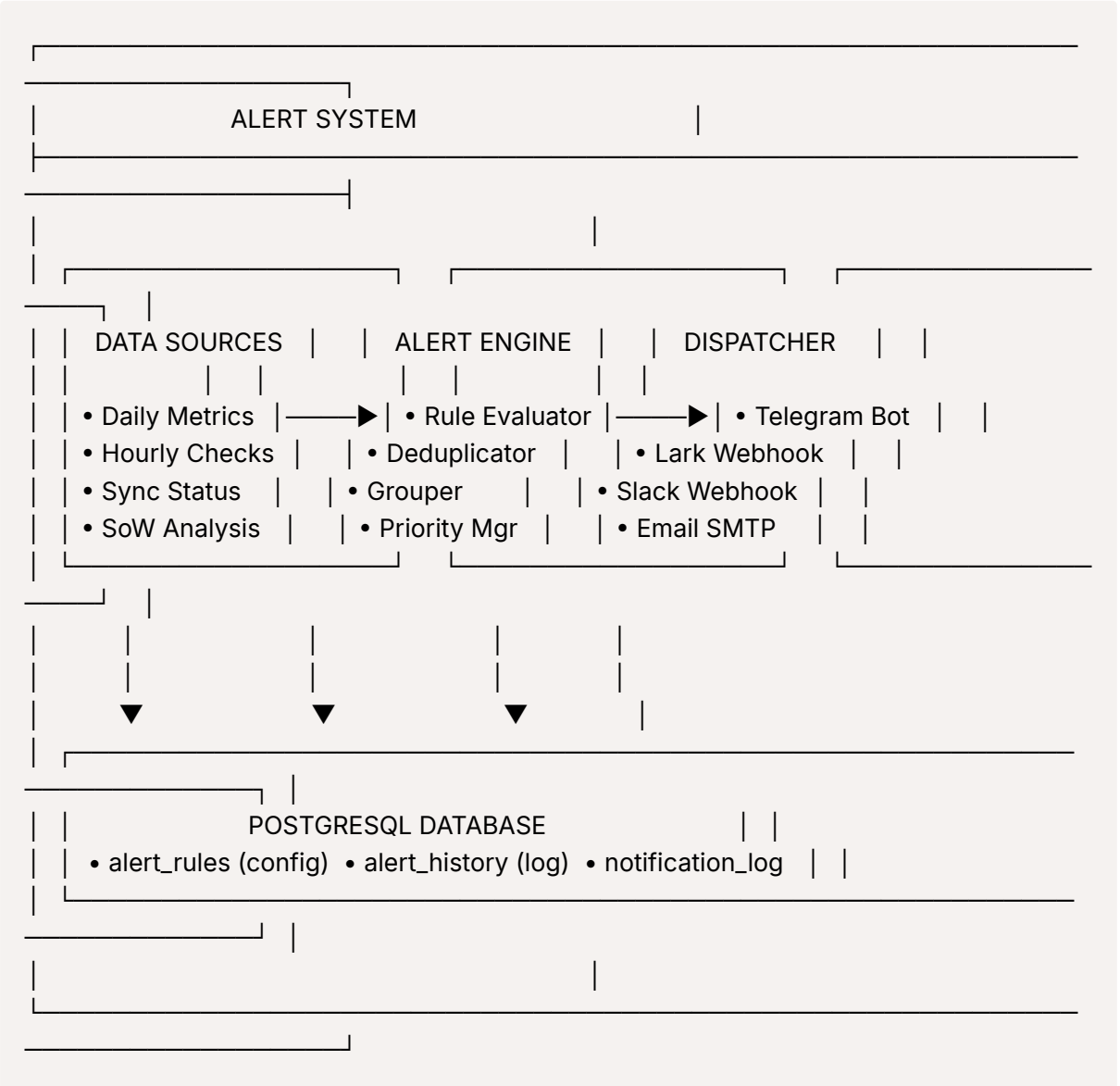
4. **Priority Routing**: HIGH → tất cả kênh, LOW → chỉ dashboard

1.3 Alert Categories

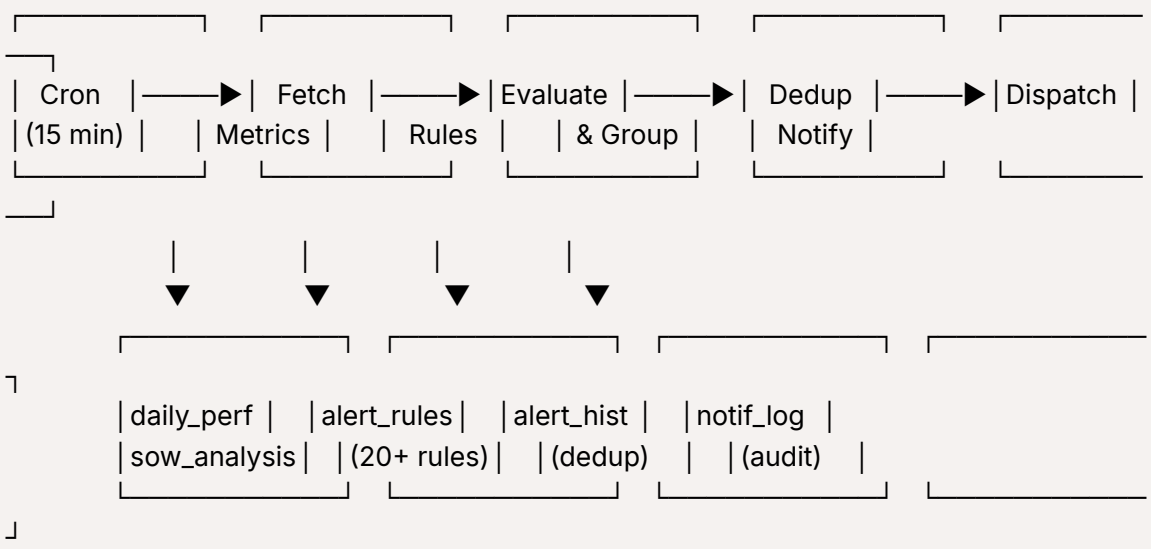
Category	Mô tả	Số Rules
REVENUE	Theo dõi doanh thu	5 rules
PERFORMANCE	eCPM, Fill Rate, Match Rate	5 rules
WATERFALL	SoW distribution, dead lines	5 rules
SYSTEM	Sync status, API errors	5 rules
RECOMMENDATION	Pending actions	4 rules

2. KIẾN TRÚC ALERT SYSTEM

2.1 Component Diagram



2.2 Data Flow



3. ALERT RULES CATALOG

3.1 REVENUE ALERTS (REV-xxx)

REV-001: Daily Revenue Drop

Attribute	Value
Code	REV-001
Name	Daily Revenue Drop
Category	REVENUE
Priority	🔴 HIGH
Condition	`revenue_dod_percent < -10%`
Metric	`estimated_earnings`
Comparison	Day-over-Day percentage
Cooldown	4 hours
Channels	Telegram, Lark, Slack, Email

SQL Logic:

```
WITH today AS (
    SELECT app_id, SUM(estimated_earnings_micros) as revenue
    FROM daily_performance
    WHERE date = CURRENT_DATE - 1
    GROUP BY app_id
),
yesterday AS (
```

```
    SELECT app_id, SUM(estimated_earnings_micros) as revenue
    FROM daily_performance
    WHERE date = CURRENT_DATE - 2
    GROUP BY app_id
)
SELECT
    t.app_id,
    t.revenue as today_revenue,
    y.revenue as yesterday_revenue,
    ((t.revenue - y.revenue)::decimal / NULLIF(y.revenue, 0) * 100) as dod_percent
FROM today t
JOIN yesterday y ON t.app_id = y.app_id
WHERE ((t.revenue - y.revenue)::decimal / NULLIF(y.revenue, 0) * 100) < -10;
```

REV-002: Weekly Revenue Drop

Attribute	Value
Code	REV-002
Name	Weekly Revenue Drop
Category	REVENUE
Priority	🔴 HIGH
Condition	`revenue_wow_percent < -15%`
Metric	`estimated_earnings`
Comparison	Week-over-Week percentage
Cooldown	8 hours
Channels	Telegram, Lark, Slack, Email

SQL Logic:

```
WITH this_week AS (
    SELECT app_id, SUM(estimated_earnings_micros) as revenue
    FROM daily_performance
    WHERE date BETWEEN CURRENT_DATE - 7 AND CURRENT_DATE - 1
    GROUP BY app_id
),
last_week AS (
    SELECT app_id, SUM(estimated_earnings_micros) as revenue
    FROM daily_performance
    WHERE date BETWEEN CURRENT_DATE - 14 AND CURRENT_DATE - 8
    GROUP BY app_id
)
SELECT
    t.app_id,
    ((t.revenue - l.revenue)::decimal / NULLIF(l.revenue, 0) * 100) as wow_percent
```

```
FROM this_week t
JOIN last_week l ON t.app_id = l.app_id
WHERE ((t.revenue - l.revenue)::decimal / NULLIF(l.revenue, 0) * 100) < -15;
```

REV-003: Revenue Spike (Unusual Increase)

Attribute	Value
Code	REV-003
Name	Revenue Spike
Category	REVENUE
Priority	🟡 MEDIUM
Condition	`revenue_dod_percent > +30%`
Metric	`estimated_earnings`
Comparison	Day-over-Day percentage
Cooldown	8 hours
Channels	Telegram, Lark

Note: Spike có thể là tín hiệu tốt nhưng cũng có thể là data error, cần verify.

REV-004: Zero Revenue Alert

Attribute	Value
Code	REV-004
Name	Zero Revenue
Category	REVENUE
Priority	🔴 HIGH
Condition	`daily_revenue = 0 AND previous_day_revenue > 0`
Metric	`estimated_earnings`
Comparison	Absolute value
Cooldown	4 hours
Channels	Telegram, Lark, Slack, Email

SQL Logic:

```
WITH today AS (
    SELECT app_id, SUM(estimated_earnings_micros) as revenue
    FROM daily_performance
    WHERE date = CURRENT_DATE - 1
    GROUP BY app_id
),
yesterday AS (
    SELECT app_id, SUM(estimated_earnings_micros) as revenue
    FROM daily_performance
```

```
    WHERE date = CURRENT_DATE - 2
    GROUP BY app_id
)
SELECT t.app_id
FROM today t
JOIN yesterday y ON t.app_id = y.app_id
WHERE t.revenue = 0 AND y.revenue > 100000; -- > $0.10
```

REV-005: Below Daily Target

Attribute	Value
Code	REV-005
Name	Below Daily Target
Category	REVENUE
Priority	🟡 MEDIUM
Condition	`daily_revenue < target_revenue`
Metric	`estimated_earnings`
Comparison	Against configured target
Cooldown	24 hours
Channels	Telegram, Lark

3.2 PERFORMANCE ALERTS (PER-xxx)

PER-001: eCPM Drop

Attribute	Value
Code	PER-001
Name	eCPM Drop
Category	PERFORMANCE
Priority	🔴 HIGH
Condition	`ecpm_dod_percent < -15%`
Metric	`observed_ecpm`
Comparison	Day-over-Day percentage
Cooldown	4 hours
Channels	Telegram, Lark, Slack, Email

SQL Logic:

```
WITH metrics AS (
    SELECT
        app_id,
        date,
```

```
        SUM(estimated_earnings_micros)::decimal / NULLIF(SUM(impressions), 0) * 1000 as e
cpm
    FROM daily_performance
    WHERE date >= CURRENT_DATE - 2
    GROUP BY app_id, date
)
SELECT
    t.app_id,
    t.ecpm as today_ecpm,
    y.ecpm as yesterday_ecpm,
    ((t.ecpm - y.ecpm) / NULLIF(y.ecpm, 0) * 100) as dod_percent
FROM metrics t
JOIN metrics y ON t.app_id = y.app_id AND t.date = y.date + 1
WHERE t.date = CURRENT_DATE - 1
  AND ((t.ecpm - y.ecpm) / NULLIF(y.ecpm, 0) * 100) < -15;
```

PER-002: Fill Rate Drop

Attribute	Value
Code	PER-002
Name	Fill Rate Below Threshold
Category	PERFORMANCE
Priority	🔴 HIGH
Condition	`fill_rate < 80%`
Metric	`impressions / ad_requests`
Comparison	Absolute threshold
Cooldown	4 hours
Channels	Telegram, Lark, Slack, Email

SQL Logic:

```
SELECT
    app_id,
    SUM(impressions)::decimal / NULLIF(SUM(ad_requests), 0) * 100 as fill_rate
FROM daily_performance
WHERE date = CURRENT_DATE - 1
GROUP BY app_id
HAVING SUM(impressions)::decimal / NULLIF(SUM(ad_requests), 0) * 100 < 80
   AND SUM(ad_requests) > 1000; -- Minimum traffic threshold
```

PER-003: Match Rate Drop

Attribute	Value
Code	PER-003
Name	Match Rate Below Threshold
Category	PERFORMANCE
Priority	🟡 MEDIUM
Condition	`match_rate < 50%`
Metric	`matched_requests / ad_requests`
Comparison	Absolute threshold
Cooldown	8 hours
Channels	Telegram, Lark

PER-004: Impressions Drop

Attribute	Value
Code	PER-004
Name	Impressions Drop
Category	PERFORMANCE
Priority	🟡 MEDIUM
Condition	`impressions_dod_percent < -20%`
Metric	`impressions`
Comparison	Day-over-Day percentage
Cooldown	8 hours
Channels	Telegram, Lark

PER-005: Show Rate Drop

Attribute	Value
Code	PER-005
Name	Show Rate Below Threshold
Category	PERFORMANCE
Priority	🟡 MEDIUM
Condition	`show_rate < 70%`
Metric	`impressions / matched_requests`
Comparison	Absolute threshold
Cooldown	8 hours
Channels	Telegram, Lark

3.3 WATERFALL ALERTS (WAT-xxx)

WAT-001: Single Source Dominance

Attribute	Value
Code	WAT-001
Name	Single Source Dominance
Category	WATERFALL
Priority	🟡 MEDIUM
Condition	`max_sow > 70%`
Metric	`sow_percentage`
Comparison	Absolute threshold
Cooldown	24 hours
Channels	Telegram, Lark

SQL Logic:

```
SELECT
    mediation_group_id,
    ad_source_instance_id,
    sow_percentage
FROM sow_analysis
WHERE date = CURRENT_DATE - 1
  AND sow_percentage > 70;
```

Risk: Quá phụ thuộc vào 1 ad source, nếu source đó có vấn đề sẽ ảnh hưởng lớn.

WAT-002: Dead Waterfall Line

Attribute	Value
Code	WAT-002
Name	Dead Waterfall Line
Category	WATERFALL
Priority	🟢 LOW
Condition	`sow < 0.5% for 7 consecutive days`
Metric	`sow_percentage`
Comparison	Average over 7 days
Cooldown	24 hours
Channels	Dashboard only

SQL Logic:

```
SELECT
    mediation_group_id,
    ad_source_instance_id,
    AVG(sow_percentage) as avg_sow
FROM sow_analysis
```

```
WHERE date >= CURRENT_DATE - 7
GROUP BY mediation_group_id, ad_source_instance_id
HAVING AVG(sow_percentage) < 0.5
  AND COUNT(*) >= 7;
```

WAT-003: Unbalanced Waterfall

Attribute	Value
Code	WAT-003
Name	Top 2 Sources Dominance
Category	WATERFALL
Priority	🟡 MEDIUM
Condition	`top_2_sow_sum > 80%`
Metric	`sow_percentage`
Comparison	Sum of top 2
Cooldown	24 hours
Channels	Telegram, Lark

SQL Logic:

```
WITH ranked AS (
  SELECT
    mediation_group_id,
    ad_source_instance_id,
    sow_percentage,
    ROW_NUMBER() OVER (PARTITION BY mediation_group_id ORDER BY sow_percentag
e DESC) as rn
  FROM sow_analysis
  WHERE date = CURRENT_DATE - 1
)
SELECT
  mediation_group_id,
  SUM(sow_percentage) as top_2_sow
FROM ranked
WHERE rn <= 2
GROUP BY mediation_group_id
HAVING SUM(sow_percentage) > 80;
```

WAT-004: Missing High Floor

Attribute	Value
Code	WAT-004
Name	No High Floor Configured

Attribute	Value
Category	WATERFALL
Priority	🟢 LOW
Condition	`max_floor < $5.00 AND avg_ecpm > $3.00`
Metric	`floor_price, ecpm`
Comparison	Threshold check
Cooldown	24 hours
Channels	Dashboard only

WAT-005: Too Many Dead Lines

Attribute	Value
Code	WAT-005
Name	Multiple Dead Lines
Category	WATERFALL
Priority	🟡 MEDIUM
Condition	`count(sow < 1%) > 3 in same MG`
Metric	`sow_percentage`
Comparison	Count threshold
Cooldown	24 hours
Channels	Telegram, Lark

SQL Logic:

```
SELECT
    mediation_group_id,
    COUNT(*) as dead_lines
FROM sow_analysis
WHERE date = CURRENT_DATE - 1
  AND sow_percentage < 1
GROUP BY mediation_group_id
HAVING COUNT(*) > 3;
```

3.4 SYSTEM ALERTS (SYS-xxx)

SYS-001: Sync Job Failed

Attribute	Value
Code	SYS-001
Name	Sync Job Failed
Category	SYSTEM

Attribute	Value
Priority	🔴 HIGH
Condition	`sync_status = 'FAILED'`
Metric	`sync_logs.status`
Comparison	Exact match
Cooldown	1 hour
Channels	Telegram, Lark, Slack, Email

SQL Logic:

```
SELECT *
FROM sync_logs
WHERE status = 'FAILED'
  AND started_at >= NOW() - INTERVAL '1 hour'
ORDER BY started_at DESC
LIMIT 1;
```

SYS-002: Sync Delayed

Attribute	Value
Code	SYS-002
Name	Sync Delayed
Category	SYSTEM
Priority	🟡 MEDIUM
Condition	`hours_since_last_sync > 6`
Metric	`sync_logs.completed_at`
Comparison	Time difference
Cooldown	4 hours
Channels	Telegram, Lark

SQL Logic:

```
SELECT
    sync_type,
    entity_type,
    MAX(completed_at) as last_sync,
    EXTRACT(EPOCH FROM (NOW() - MAX(completed_at))) / 3600 as hours_ago
FROM sync_logs
WHERE status = 'SUCCESS'
GROUP BY sync_type, entity_type
HAVING EXTRACT(EPOCH FROM (NOW() - MAX(completed_at))) / 3600 > 6;
```

SYS-003: API Error Rate High

Attribute	Value
Code	SYS-003
Name	API Error Rate High
Category	SYSTEM
Priority	🔴 HIGH
Condition	`api_error_rate > 5%`
Metric	`api_logs.error_count / total_count`
Comparison	Percentage threshold
Cooldown	2 hours
Channels	Telegram, Lark, Slack, Email

SYS-004: Data Freshness Issue

Attribute	Value
Code	SYS-004
Name	Stale Data
Category	SYSTEM
Priority	🔴 HIGH
Condition	`latest_data_date < TODAY - 2`
Metric	`daily_performance.date`
Comparison	Date difference
Cooldown	4 hours
Channels	Telegram, Lark, Slack, Email

SQL Logic:

```
SELECT MAX(date) as latest_date
FROM daily_performance
HAVING MAX(date) < CURRENT_DATE - 2;
```

SYS-005: Partial Sync Warning

Attribute	Value
Code	SYS-005
Name	Partial Sync
Category	SYSTEM
Priority	🟡 MEDIUM
Condition	`synced_apps_percent < 90%`
Metric	`sync_logs.records_processed`

Attribute	Value
Comparison	Percentage of expected
Cooldown	4 hours
Channels	Telegram, Lark

3.5 RECOMMENDATION ALERTS (REC-xxx)

REC-001: High Priority Pending

Attribute	Value
Code	REC-001
Name	Many High Priority Recommendations Pending
Category	RECOMMENDATION
Priority	🟡 MEDIUM
Condition	`count(priority = 'HIGH' AND status = 'PENDING') > 5`
Metric	`sow_analysis.priority`
Comparison	Count threshold
Cooldown	24 hours
Channels	Telegram, Lark

REC-002: Stale Recommendations

Attribute	Value
Code	REC-002
Name	Recommendations Not Acted
Category	RECOMMENDATION
Priority	🟢 LOW
Condition	`days_since_recommendation > 7`
Metric	`sow_analysis.calculated_at`
Comparison	Time difference
Cooldown	24 hours
Channels	Dashboard only

REC-003: Quick Win Available

Attribute	Value
Code	REC-003
Name	Quick Win Opportunity
Category	RECOMMENDATION
Priority	🟢 LOW

Attribute	Value
Condition	`estimated_impact > $100/day`
Metric	`sow_analysis.estimated_impact_micros`
Comparison	Threshold
Cooldown	24 hours
Channels	Dashboard, Daily Digest

REC-004: A/B Test Results Ready

Attribute	Value
Code	REC-004
Name	A/B Test Running > 7 Days
Category	RECOMMENDATION
Priority	🟢 LOW
Condition	`ab_test_days > 7`
Metric	`mediation_group.ab_test_state`
Comparison	Time threshold
Cooldown	24 hours
Channels	Telegram, Lark

4. DATABASE SCHEMA

4.1 alert_rules Table

```
CREATE TABLE alert_rules (
    id SERIAL PRIMARY KEY,
    code VARCHAR(20) UNIQUE NOT NULL,          -- REV-001, PER-002, etc.
    name VARCHAR(100) NOT NULL,
    category VARCHAR(50) NOT NULL,             -- REVENUE/PERFORMANCE/WATERFALL/S
YSTEM/RECOMMENDATION
    description TEXT,

    -- Condition definition
    metric VARCHAR(50) NOT NULL,               -- revenue, ecpm, fill_rate, sow_percentage, e
tc.
    operator VARCHAR(10) NOT NULL,             -- <, >, <=, >=, =, !=
    threshold DECIMAL(15,4) NOT NULL,
    comparison_type VARCHAR(30) DEFAULT 'absolute',  -- absolute, dod_percent, wow_pe
rcent, dod_absolute

    -- Alert configuration
```

```
    priority VARCHAR(10) NOT NULL DEFAULT 'MEDIUM',  -- HIGH, MEDIUM, LOW
    cooldown_hours INTEGER NOT NULL DEFAULT 4,
    is_active BOOLEAN NOT NULL DEFAULT true,

    -- Scope configuration
    apply_to VARCHAR(20) DEFAULT 'all',        -- all, specific_apps, specific_mgs
    app_ids TEXT[],                            -- Array of app_ids if specific
    mediation_group_ids TEXT[],                -- Array of mg_ids if specific

    -- Notification channels (which channels to send)
    notify_telegram BOOLEAN DEFAULT true,
    notify_lark BOOLEAN DEFAULT true,
    notify_slack BOOLEAN DEFAULT false,
    notify_email BOOLEAN DEFAULT false,

    -- Metadata
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    created_by VARCHAR(100),

    -- Constraints
    CONSTRAINT valid_priority CHECK (priority IN ('HIGH', 'MEDIUM', 'LOW')),
    CONSTRAINT valid_category CHECK (category IN ('REVENUE', 'PERFORMANCE', 'WATE
RFALL', 'SYSTEM', 'RECOMMENDATION')),
    CONSTRAINT valid_operator CHECK (operator IN ('<', '>', '<=', '>=', '=', '!='))
);

-- Indexes
CREATE INDEX idx_alert_rules_category ON alert_rules(category);
CREATE INDEX idx_alert_rules_active ON alert_rules(is_active);
CREATE INDEX idx_alert_rules_priority ON alert_rules(priority);
```

4.2 alert_history Table

```
CREATE TABLE alert_history (
    id BIGSERIAL PRIMARY KEY,
    rule_id INTEGER NOT NULL REFERENCES alert_rules(id),
    rule_code VARCHAR(20) NOT NULL,            -- Denormalized for quick lookup

    -- Context
    app_id VARCHAR(100),
    app_name VARCHAR(255),
    mediation_group_id VARCHAR(100),
    mediation_group_name VARCHAR(255),
```

```
    -- Alert details
    triggered_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT NOW(),
    metric_value DECIMAL(15,4) NOT NULL,
    threshold_value DECIMAL(15,4) NOT NULL,
    comparison_value DECIMAL(15,4),           -- Previous value for DoD/WoW
    message TEXT NOT NULL,
    details JSONB,                            -- Additional context

    -- Status tracking
    status VARCHAR(20) NOT NULL DEFAULT 'PENDING',  -- PENDING, ACKNOWLEDGED, R
ESOLVED, SUPPRESSED
    acknowledged_at TIMESTAMP WITH TIME ZONE,
    acknowledged_by VARCHAR(100),
    resolved_at TIMESTAMP WITH TIME ZONE,
    resolved_by VARCHAR(100),
    resolution_notes TEXT,

    -- Notification tracking
    telegram_sent BOOLEAN DEFAULT false,
    telegram_sent_at TIMESTAMP WITH TIME ZONE,
    telegram_message_id VARCHAR(100),

    lark_sent BOOLEAN DEFAULT false,
    lark_sent_at TIMESTAMP WITH TIME ZONE,
    lark_message_id VARCHAR(100),

    slack_sent BOOLEAN DEFAULT false,
    slack_sent_at TIMESTAMP WITH TIME ZONE,
    slack_message_id VARCHAR(100),

    email_sent BOOLEAN DEFAULT false,
    email_sent_at TIMESTAMP WITH TIME ZONE,

    -- Metadata
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    CONSTRAINT valid_status CHECK (status IN ('PENDING', 'ACKNOWLEDGED', 'RESOLVE
D', 'SUPPRESSED'))
);

-- Indexes for performance
CREATE INDEX idx_alert_history_rule_id ON alert_history(rule_id);
CREATE INDEX idx_alert_history_app_id ON alert_history(app_id);
CREATE INDEX idx_alert_history_mg_id ON alert_history(mediation_group_id);
CREATE INDEX idx_alert_history_triggered ON alert_history(triggered_at DESC);
CREATE INDEX idx_alert_history_status ON alert_history(status);
```

```
CREATE INDEX idx_alert_history_rule_app_time ON alert_history(rule_id, app_id, triggered_a
t DESC);
```

4.3 notification_channels Table

```
CREATE TABLE notification_channels (
    id SERIAL PRIMARY KEY,
    channel_type VARCHAR(20) NOT NULL,          -- telegram, lark, slack, email
    name VARCHAR(100) NOT NULL,
    description TEXT,

    -- Channel-specific configuration (stored as JSON)
    config JSONB NOT NULL,
    /*
    Telegram: { "bot_token": "xxx", "chat_id": "-123456" }
    Lark: { "webhook_url": "https://..." }
    Slack: { "webhook_url": "https://..." }
    Email: { "smtp_host": "...", "smtp_port": 587, "recipients": ["a@b.com"] }
    */

    -- Routing configuration
    min_priority VARCHAR(10) DEFAULT 'LOW',     -- Only send alerts >= this priority
    is_active BOOLEAN DEFAULT true,

    -- Metadata
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    CONSTRAINT valid_channel_type CHECK (channel_type IN ('telegram', 'lark', 'slack', 'em
ail'))
);
```

4.4 Initial Data - Alert Rules

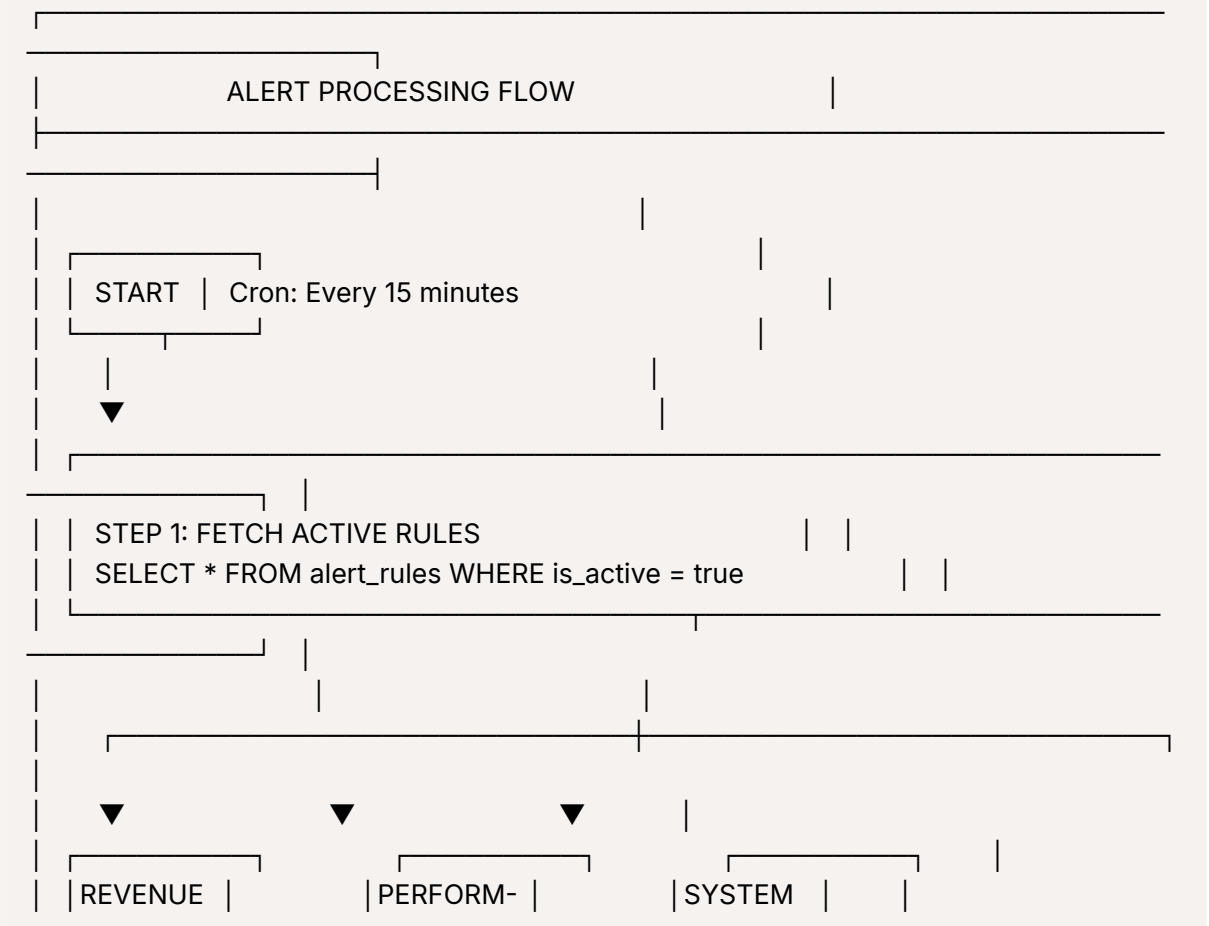
```
-- Revenue Alerts
INSERT INTO alert_rules (code, name, category, metric, operator, threshold, comparison_ty
pe, priority, cooldown_hours, notify_telegram, notify_lark, notify_slack, notify_email) VALUE
S
('REV-001', 'Daily Revenue Drop', 'REVENUE', 'revenue', '<', -10, 'dod_percent', 'HIGH', 4, tr
ue, true, true, true),
('REV-002', 'Weekly Revenue Drop', 'REVENUE', 'revenue', '<', -15, 'wow_percent', 'HIGH',
8, true, true, true, true),
('REV-003', 'Revenue Spike', 'REVENUE', 'revenue', '>', 30, 'dod_percent', 'MEDIUM', 8, tru
e, true, false, false),
```

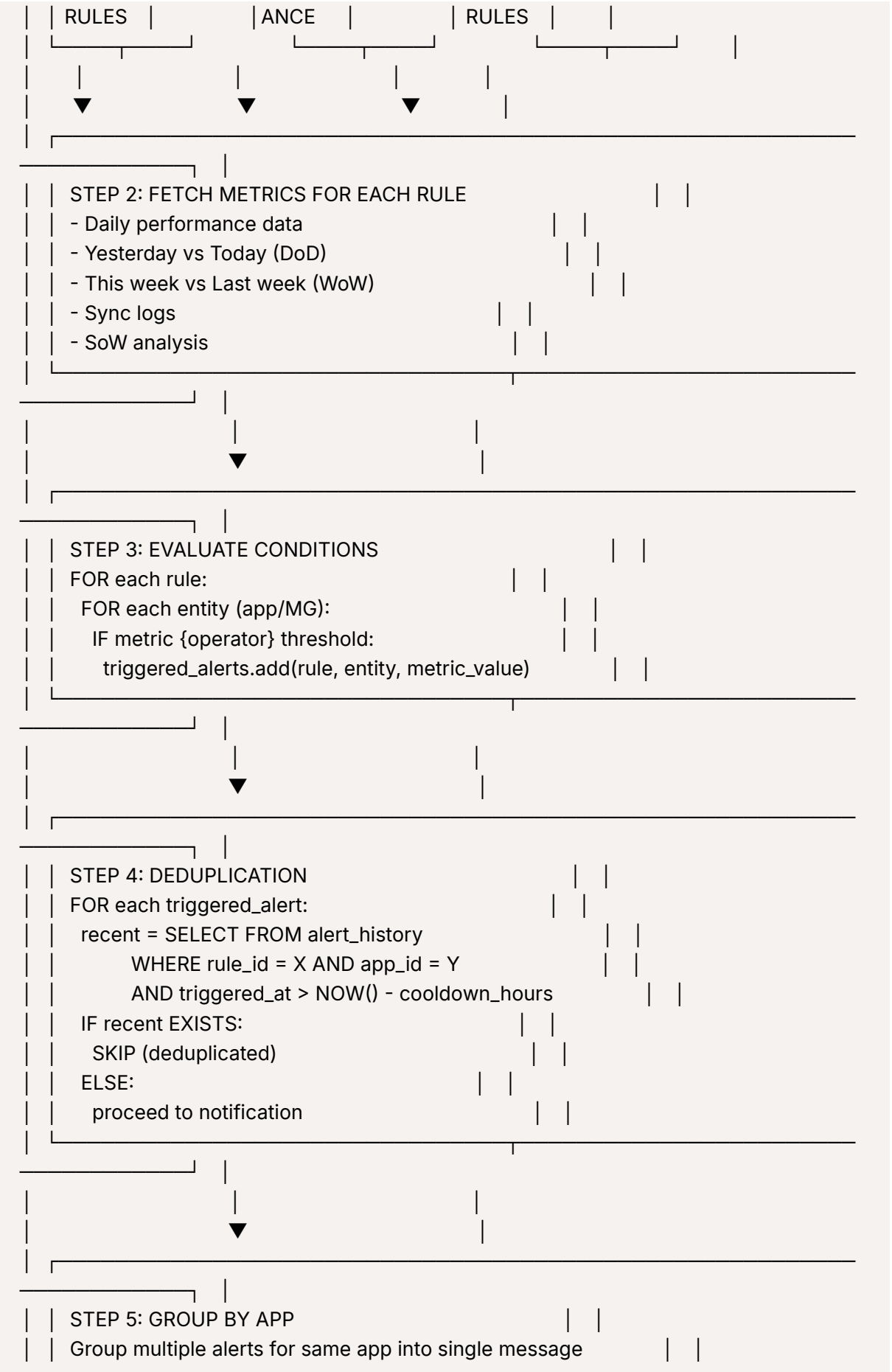
```
('REV-004', 'Zero Revenue', 'REVENUE', 'revenue', '=', 0, 'absolute', 'HIGH', 4, true, true, tru
e, true),
('REV-005', 'Below Daily Target', 'REVENUE', 'revenue', '<', 0, 'absolute', 'MEDIUM', 24, tru
e, true, false, false);

-- Performance Alerts
INSERT INTO alert_rules (code, name, category, metric, operator, threshold, comparison_ty
pe, priority, cooldown_hours, notify_telegram, notify_lark, notify_slack, notify_email) VALUE
S
('PER-001', 'eCPM Drop', 'PERFORMANCE', 'ecpm', '<', -15, 'dod_percent', 'HIGH', 4, true, tr
ue, true, true),
('PER-002', 'Fill Rate Low', 'PERFORMANCE', 'fill_rate', '<', 80, 'absolute', 'HIGH', 4, true, tru
e, true, true),
('PER-003', 'Match Rate Low', 'PERFORMANCE', 'match_rate', '<', 50, 'absolute', 'MEDIUM',
8, true, true, false, false),
('PER-004', 'Impressions Drop', 'PERFORMANCE', 'impressions', '<', -20, 'dod_percent', 'M
EDIUM', 8, true, true, false, false),
('PER-005', 'Show Rate Low', 'PERFORMANCE', 'show_rate', '<', 70, 'absolute', 'MEDIUM',
8, true, true, false, false);

-- Waterfall Alerts
INSERT INTO alert_rules (code, name, category, metric, operator, threshold, comparison_ty
pe, priority, cooldown_hours, notify_telegram, notify_lark, notify_slack, notify_email) VALUE
S
('WAT-001', 'Single Source Dominance', 'WATERFALL', 'sow_percentage', '>', 70, 'absolute',
'MEDIUM', 24, true, true, false, false),
('WAT-002', 'Dead Waterfall Line', 'WATERFALL', 'sow_percentage', '<', 0.5, 'absolute', 'LO
W', 24, false, false, false, false),
('WAT-003', 'Top 2 Sources Dominance', 'WATERFALL', 'top_2_sow', '>', 80, 'absolute', 'ME
DIUM', 24, true, true, false, false),
('WAT-004', 'No High Floor', 'WATERFALL', 'max_floor', '<', 5000000, 'absolute', 'LOW', 24,
false, false, false, false),
('WAT-005', 'Too Many Dead Lines', 'WATERFALL', 'dead_line_count', '>', 3, 'absolute', 'ME
DIUM', 24, true, true, false, false);

-- System Alerts
INSERT INTO alert_rules (code, name, category, metric, operator, threshold, comparison_ty
pe, priority, cooldown_hours, notify_telegram, notify_lark, notify_slack, notify_email) VALUE
S
('SYS-001', 'Sync Job Failed', 'SYSTEM', 'sync_status', '=', 0, 'absolute', 'HIGH', 1, true, tru
e, true, true),
('SYS-002', 'Sync Delayed', 'SYSTEM', 'hours_since_sync', '>', 6, 'absolute', 'MEDIUM', 4, t
rue, true, false, false),
('SYS-003', 'API Error Rate High', 'SYSTEM', 'api_error_rate', '>', 5, 'absolute', 'HIGH', 2, tru
e, true, true, true),
('SYS-004', 'Stale Data', 'SYSTEM', 'data_age_days', '>', 2, 'absolute', 'HIGH', 4, true, true, t
```

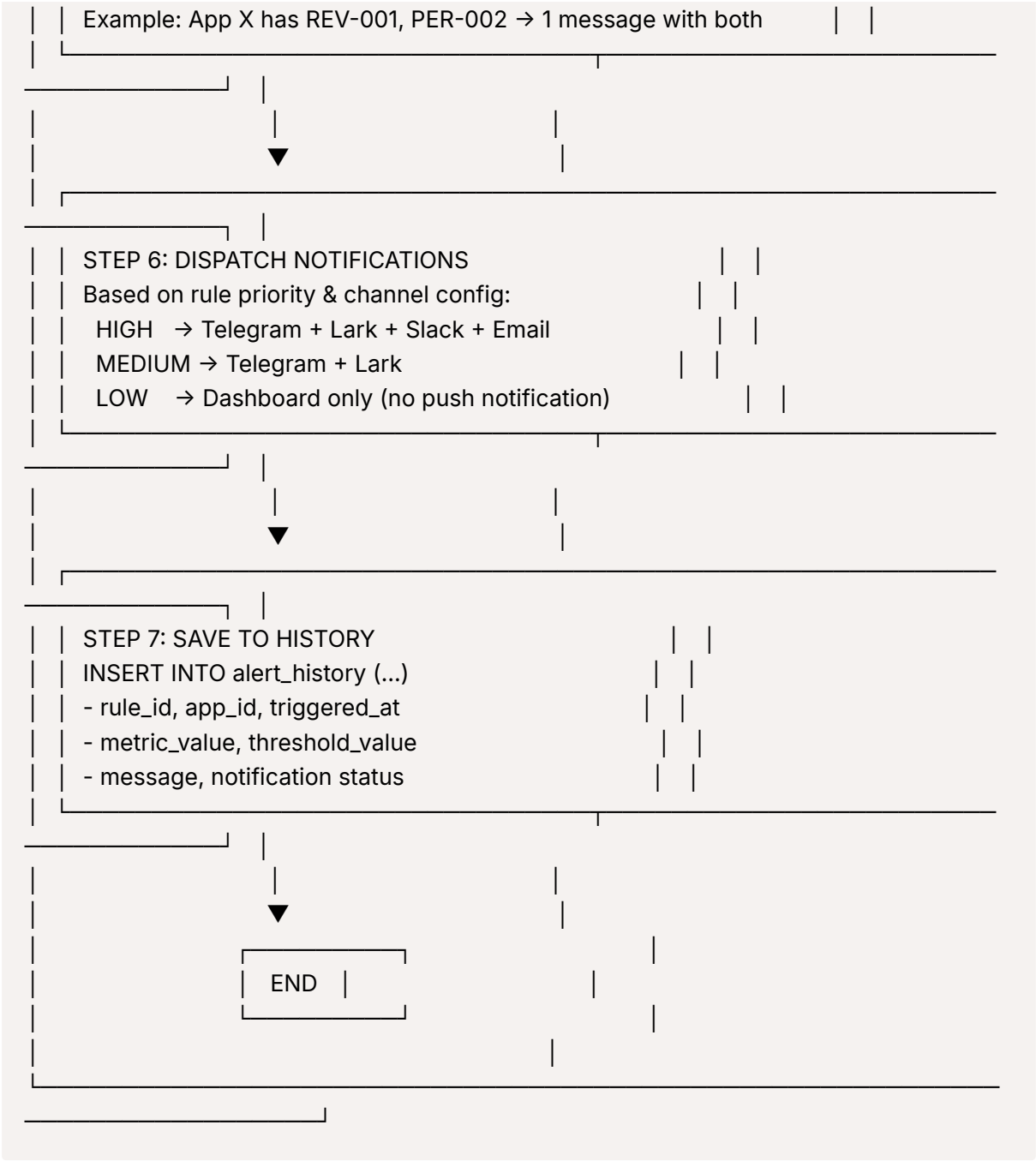
```
rue, true),
('SYS-005', 'Partial Sync', 'SYSTEM', 'sync_completeness', '<', 90, 'absolute', 'MEDIUM', 4,
true, true, false, false);

-- Recommendation Alerts
INSERT INTO alert_rules (code, name, category, metric, operator, threshold, comparison_ty
pe, priority, cooldown_hours, notify_telegram, notify_lark, notify_slack, notify_email) VALUE
S
('REC-001', 'High Priority Pending', 'RECOMMENDATION', 'high_priority_count', '>', 5, 'abso
lute', 'MEDIUM', 24, true, true, false, false),
('REC-002', 'Stale Recommendations', 'RECOMMENDATION', 'recommendation_age_days',
'>', 7, 'absolute', 'LOW', 24, false, false, false, false),
('REC-003', 'Quick Win Available', 'RECOMMENDATION', 'estimated_impact', '>', 10000000
0, 'absolute', 'LOW', 24, false, false, false, false),
('REC-004', 'A/B Test Results Ready', 'RECOMMENDATION', 'ab_test_days', '>', 7, 'absolut
e', 'LOW', 24, true, true, false, false);
```

5. ALERT PROCESSING FLOW

5.1 Main Flow




RULES
ANCE
RULES
STEP 2: FETCH METRICS FOR EACH RULE
- Daily performance data
- Yesterday vs Today (DoD)
- This week vs Last week (WoW)
- Sync logs
- SoW analysis
STEP 3: EVALUATE CONDITIONS
FOR each rule:
FOR each entity (app/MG):
IF metric {operator} threshold:
triggered_alerts.add(rule, entity, metric_value)
STEP 4: DEDUPLICATION
FOR each triggered_alert:
recent = SELECT FROM alert_history
WHERE rule_id = X AND app_id = Y
AND triggered_at > NOW() - cooldown_hours
IF recent EXISTS:
SKIP (deduplicated)
ELSE:
proceed to notification
STEP 5: GROUP BY APP
Group multiple alerts for same app into single message



5.2 Deduplication Logic

```
Cooldown Periods:
├── HIGH Priority   → 4 hours
├── MEDIUM Priority → 8 hours
└── LOW Priority    → 24 hours

Check:
SELECT COUNT(*) FROM alert_history
WHERE rule_id = {rule_id}
  AND (app_id = {app_id} OR app_id IS NULL)
```

```
  AND triggered_at > NOW() - INTERVAL '{cooldown} hours'
  AND status != 'RESOLVED'

IF count > 0 → SKIP (already alerted)
ELSE → PROCEED
```

5.3 Priority Routing Matrix

Priority	Telegram	Lark	Slack	Email	Dashboard
🔴 HIGH	✅	✅	✅	✅	✅
🟡 MEDIUM	✅	✅	❌	❌	✅
🟢 LOW	❌	❌	❌	❌	✅

6. NOTIFICATION CHANNELS

6.1 Telegram Bot Setup

```
{
  "channel_type": "telegram",
  "config": {
    "bot_token": "123456789:ABCdefGHIjklMNOpqrsTUVwxyz",
    "chat_id": "-1001234567890",
    "parse_mode": "HTML",
    "disable_web_page_preview": true
  }
}
```

API Endpoint:

```
POST https://api.telegram.org/bot{token}/sendMessage
Content-Type: application/json

{
  "chat_id": "{chat_id}",
  "text": "{message}",
  "parse_mode": "HTML",
  "disable_web_page_preview": true
}
```

6.2 Lark Bot Setup

```
{
  "channel_type": "lark",
  "config": {
    "webhook_url": "https://open.larksuite.com/open-apis/bot/v2/hook/xxxx",
    "secret": "optional_sign_key"
  }
}
```

API Endpoint:

```
POST {webhook_url}
Content-Type: application/json

{
  "msg_type": "interactive",
  "card": {
    "config": { "wide_screen_mode": true },
    "header": {
      "title": { "tag": "plain_text", "content": "🔴 Alert Title" },
      "template": "red"
    },
    "elements": [
      {
        "tag": "div",
        "text": { "tag": "lark_md", "content": "**App:** WeatherPlus\n**Issue:** Revenue drop
-18%" }
      }
    ]
  }
}
```

6.3 Slack Webhook Setup

```
{
  "channel_type": "slack",
  "config": {
    "webhook_url": "https://hooks.slack.com/services/T00000000/B00000000/XXXXXXXX
XXXXXXXXXXXXXXXX",
    "channel": "#alerts-mediation"
  }
}
```

API Endpoint:

```
POST {webhook_url}
Content-Type: application/json

{
  "channel": "#alerts-mediation",
  "username": "Mediation Pro",
  "icon_emoji": ":warning:",
  "attachments": [
    {
      "color": "#ff0000",
      "title": "🔴 Revenue Drop Alert",
      "text": "App: WeatherPlus\nIssue: Daily revenue dropped -18.5%",
      "fields": [
        { "title": "Yesterday", "value": "$1,245", "short": true },
        { "title": "Today", "value": "$1,015", "short": true }
      ],
      "footer": "Mediation Pro Alert System",
      "ts": 1234567890
    }
  ]
}
```

6.4 Email SMTP Setup

```
{
  "channel_type": "email",
  "config": {
    "smtp_host": "smtp.gmail.com",
    "smtp_port": 587,
    "use_tls": true,
    "username": "alerts@company.com",
    "password": "app_password",
    "from_address": "Mediation Pro <alerts@company.com>",
    "recipients": ["team-lead@company.com", "marketing@company.com"]
  }
}
```

---

7. MESSAGE TEMPLATES

7.1 HIGH Priority Alert Template

Telegram/Lark Format (Markdown)

🔴 HIGH ALERT: {rule_name}
━━━━━━━━━━━━━━━━━━━━

📱 App: {app_name}
{app_id}

📉 Issue: {description}

📊 Metrics:
• Current: {metric_value}
• Threshold: {threshold_value}
• Change: {change_percent}%

📍 Affected:
{affected_items}

💡 Recommended Actions:
{recommendations}

🔗 View Dashboard
⏰ {timestamp} UTC

Example Output

🔴 HIGH ALERT: Daily Revenue Drop
━━━━━━━━━━━━━━━━━━━━

📱 App: WeatherPlus Pro
ca-app-pub-1234567890~9876543210

📉 Issue: Daily revenue dropped -18.5%

📊 Metrics:
• Yesterday: $1,245.00
• Today: $1,015.30
• Change: -18.5%

📍 Affected:
• Banner US (-22%)
• Interstitial Global (-15%)

💡 Recommended Actions:
• Check eCPM changes in top waterfall lines
• Review fill rate by ad source
• Verify no technical issues with SDK

🔗 View Dashboard
⏰ 2025-01-15 09:15 UTC

7.2 MEDIUM Priority Alert Template

```
🟡 <b>{rule_name}</b>
━━━━━━━━━━━━━━━━━━━━

📱 <b>App:</b> {app_name}
⚠️ <b>Issue:</b> {description}
📊 <b>Value:</b> {metric_value} (threshold: {threshold_value})

🔗 <a href="{dashboard_url}">View Details</a>
```

7.3 Daily Summary Template

```
📊 <b>DAILY ALERT SUMMARY</b>
━━━━━━━━━━━━━━━━━━━━

📅 {date}

🔴 <b>HIGH:</b> {high_count} alerts
🟡 <b>MEDIUM:</b> {medium_count} alerts
🟢 <b>LOW:</b> {low_count} alerts

📈 <b>Top Issues:</b>
{top_issues_list}

💡 <b>Pending Actions:</b> {pending_count} recommendations

🔗 <a href="{dashboard_url}">Open Dashboard</a>
```

---

8. CONFIGURATION

8.1 appsettings.json

```
{
  "AlertSystem": {
    "Enabled": true,
    "CheckIntervalMinutes": 15,
    "DefaultCooldownHours": 4,
    "MaxAlertsPerRun": 100,
```

```
  "GroupAlertsByApp": true,

  "Channels": {
    "Telegram": {
      "Enabled": true,
      "BotToken": "${TELEGRAM_BOT_TOKEN}",
      "DefaultChatId": "${TELEGRAM_CHAT_ID}"
    },
    "Lark": {
      "Enabled": true,
      "WebhookUrl": "${LARK_WEBHOOK_URL}"
    },
    "Slack": {
      "Enabled": false,
      "WebhookUrl": "${SLACK_WEBHOOK_URL}"
    },
    "Email": {
      "Enabled": false,
      "SmtpHost": "smtp.gmail.com",
      "SmtpPort": 587,
      "UseTls": true,
      "Username": "${EMAIL_USERNAME}",
      "Password": "${EMAIL_PASSWORD}",
      "FromAddress": "Mediation Pro <alerts@company.com>",
      "Recipients": ["team@company.com"]
    }
  },

  "Thresholds": {
    "MinTrafficForAlerts": 1000,
    "MinRevenueForAlerts": 100000
  },

  "DailySummary": {
    "Enabled": true,
    "SendTime": "09:00",
    "TimeZone": "Asia/Ho_Chi_Minh"
  }
 }
}
```

8.2 Environment Variables

```
# Telegram
TELEGRAM_BOT_TOKEN=123456789:ABCdefGHIjkIMNOpqrsTUVwxyz
```

```
TELEGRAM_CHAT_ID=-1001234567890

# Lark
LARK_WEBHOOK_URL=https://open.larksuite.com/open-apis/bot/v2/hook/xxxx

# Slack (optional)
SLACK_WEBHOOK_URL=https://hooks.slack.com/services/xxx/yyy/zzz

# Email (optional)
EMAIL_USERNAME=alerts@company.com
EMAIL_PASSWORD=app_specific_password
```

---

9. API ENDPOINTS

9.1 Alert Rules Management

List All Rules

```
GET /api/alerts/rules
Authorization: Bearer {token}

Response:
{
  "rules": [
    {
      "id": 1,
      "code": "REV-001",
      "name": "Daily Revenue Drop",
      "category": "REVENUE",
      "priority": "HIGH",
      "isActive": true,
      "threshold": -10,
      "comparisonType": "dod_percent",
      "cooldownHours": 4
    }
  ],
  "total": 24
}
```

Update Rule

```
PATCH /api/alerts/rules/{id}
Authorization: Bearer {token}
Content-Type: application/json
```

```
{
  "threshold": -15,
  "isActive": true,
  "cooldownHours": 6
}
```

Toggle Rule

```
POST /api/alerts/rules/{id}/toggle
Authorization: Bearer {token}
```

9.2 Alert History

List Recent Alerts

```
GET /api/alerts/history?page=1&pageSize=20&status=PENDING
Authorization: Bearer {token}

Response:
{
  "alerts": [
    {
      "id": 123,
      "ruleCode": "REV-001",
      "ruleName": "Daily Revenue Drop",
      "appId": "ca-app-pub-xxx~yyy",
      "appName": "WeatherPlus",
      "triggeredAt": "2025-01-15T09:15:00Z",
      "metricValue": -18.5,
      "thresholdValue": -10,
      "status": "PENDING",
      "message": "Daily revenue dropped -18.5%"
    }
  ],
  "total": 45,
  "page": 1,
  "pageSize": 20
}
```

Acknowledge Alert

```
POST /api/alerts/history/{id}/acknowledge
Authorization: Bearer {token}
```

```
Content-Type: application/json

{
  "note": "Investigating the issue"
}
```

Resolve Alert

```
POST /api/alerts/history/{id}/resolve
Authorization: Bearer {token}
Content-Type: application/json

{
  "note": "Fixed by adjusting waterfall floors"
}
```

9.3 Manual Alert Trigger

Trigger Alert Check

```
POST /api/alerts/check
Authorization: Bearer {token}

Response:
{
  "triggered": 3,
  "skipped": 12,
  "errors": 0,
  "details": [
    { "ruleCode": "REV-001", "appId": "xxx", "sent": true },
    { "ruleCode": "PER-002", "appId": "yyy", "sent": true }
  ]
}
```

Test Notification Channel

```
POST /api/alerts/test-channel
Authorization: Bearer {token}
Content-Type: application/json

{
  "channel": "telegram",
  "message": "Test message from Mediation Pro"
}
```

10. CODE IMPLEMENTATION

10.1 AlertRule Entity

```
public class AlertRule
{
    public int Id { get; set; }
    public string Code { get; set; } = string.Empty;
    public string Name { get; set; } = string.Empty;
    public string Category { get; set; } = string.Empty;
    public string? Description { get; set; }

    // Condition
    public string Metric { get; set; } = string.Empty;
    public string Operator { get; set; } = string.Empty;
    public decimal Threshold { get; set; }
    public string ComparisonType { get; set; } = "absolute";

    // Configuration
    public string Priority { get; set; } = "MEDIUM";
    public int CooldownHours { get; set; } = 4;
    public bool IsActive { get; set; } = true;

    // Scope
    public string ApplyTo { get; set; } = "all";
    public List<string>? AppIds { get; set; }
    public List<string>? MediationGroupIds { get; set; }

    // Channels
    public bool NotifyTelegram { get; set; } = true;
    public bool NotifyLark { get; set; } = true;
    public bool NotifySlack { get; set; } = false;
    public bool NotifyEmail { get; set; } = false;
}
```

10.2 IAlertService Interface

```
public interface IAlertService
{
    ///<summary>
    /// Run alert evaluation for all active rules
    ///</summary>
    Task<AlertCheckResult> EvaluateAllRulesAsync(CancellationToken cancellationToken =
```

```
default);

    ///<summary>
    /// Evaluate a specific rule
    ///</summary>
    Task<IEnumerable<AlertHistory>> EvaluateRuleAsync(AlertRule rule, CancellationToken
cancellationToken = default);

    ///<summary>
    /// Check if alert should be deduplicated
    ///</summary>
    Task<bool> ShouldDeduplicateAsync(int ruleId, string? appId, string? mgId);

    ///<summary>
    /// Send notifications for triggered alerts
    ///</summary>
    Task SendNotificationsAsync(IEnumerable<AlertHistory> alerts, CancellationToken canc
ellationToken = default);

    ///<summary>
    /// Acknowledge an alert
    ///</summary>
    Task AcknowledgeAlertAsync(long alertId, string acknowledgedBy, string? note = null);

    ///<summary>
    /// Resolve an alert
    ///</summary>
    Task ResolveAlertAsync(long alertId, string resolvedBy, string? note = null);
}

public record AlertCheckResult(
    int TriggeredCount,
    int SkippedCount,
    int ErrorCount,
    IEnumerable<AlertDetail> Details
);

public record AlertDetail(
    string RuleCode,
    string? AppId,
    bool Sent,
    string? Error
);
```

10.3 AlertService Implementation (Core Logic)

```
public class AlertService : IAlertService
{
    private readonly IAlertRuleRepository _ruleRepo;
    private readonly IAlertHistoryRepository _historyRepo;
    private readonly IDailyPerformanceRepository _perfRepo;
    private readonly ISoWAnalysisRepository _sowRepo;
    private readonly INotificationService _notificationService;
    private readonly ILogger<AlertService> _logger;

    public async Task<AlertCheckResult> EvaluateAllRulesAsync(CancellationToken cancell
ationToken = default)
    {
        var rules = await _ruleRepo.GetActiveRulesAsync();
        var triggered = new List<AlertHistory>();
        var skipped = 0;
        var errors = 0;
        var details = new List<AlertDetail>();

        foreach (var rule in rules)
        {
            try
            {
                var alerts = await EvaluateRuleAsync(rule, cancellationToken);

                foreach (var alert in alerts)
                {
                    // Check deduplication
                    if (await ShouldDeduplicateAsync(rule.Id, alert.AppId, alert.MediationGroupId))
                    {
                        skipped++;
                        details.Add(new AlertDetail(rule.Code, alert.AppId, false, "Deduplicated"));
                        continue;
                    }

                    triggered.Add(alert);
                }
            }
            catch (Exception ex)
            {
                errors++;
                _logger.LogError(ex, "Error evaluating rule {RuleCode}", rule.Code);
                details.Add(new AlertDetail(rule.Code, null, false, ex.Message));
            }
        }
```

```
        // Send notifications
        if (triggered.Any())
        {
            await SendNotificationsAsync(triggered, cancellationToken);

            foreach (var alert in triggered)
            {
                details.Add(new AlertDetail(alert.RuleCode, alert.AppId, true, null));
            }
        }

        return new AlertCheckResult(triggered.Count, skipped, errors, details);
    }

    public async Task<IEnumerable<AlertHistory>> EvaluateRuleAsync(AlertRule rule, Canc
ellationToken cancellationToken = default)
    {
        var alerts = new List<AlertHistory>();

        switch (rule.Category)
        {
            case "REVENUE":
                alerts.AddRange(await EvaluateRevenueRuleAsync(rule));
                break;
            case "PERFORMANCE":
                alerts.AddRange(await EvaluatePerformanceRuleAsync(rule));
                break;
            case "WATERFALL":
                alerts.AddRange(await EvaluateWaterfallRuleAsync(rule));
                break;
            case "SYSTEM":
                alerts.AddRange(await EvaluateSystemRuleAsync(rule));
                break;
            case "RECOMMENDATION":
                alerts.AddRange(await EvaluateRecommendationRuleAsync(rule));
                break;
        }

        return alerts;
    }

    private async Task<IEnumerable<AlertHistory>> EvaluateRevenueRuleAsync(AlertRule r
ule)
    {
        var alerts = new List<AlertHistory>();
        var today = DateOnly.FromDateTime(DateTime.UtcNow.AddDays(-1));
```

```
        var yesterday = today.AddDays(-1);

        // Get metrics based on comparison type
        if (rule.ComparisonType == "dod_percent")
        {
            var todayData = await _perfRepo.GetByDateRangeAsync(today, today);
            var yesterdayData = await _perfRepo.GetByDateRangeAsync(yesterday, yesterday);

            var todayByApp = todayData.GroupBy(p => p.AppId)
                .ToDictionary(g => g.Key, g => g.Sum(p => p.EstimatedEarningsMicros));

            var yesterdayByApp = yesterdayData.GroupBy(p => p.AppId)
                .ToDictionary(g => g.Key, g => g.Sum(p => p.EstimatedEarningsMicros));

            foreach (var (appId, todayRevenue) in todayByApp)
            {
                if (!yesterdayByApp.TryGetValue(appId, out var yesterdayRevenue) || yesterdayR
evenue == 0)
                    continue;

                var dodPercent = ((decimal)(todayRevenue - yesterdayRevenue) / yesterdayReve
nue) * 100;

                if (EvaluateCondition(dodPercent, rule.Operator, rule.Threshold))
                {
                    alerts.Add(CreateAlert(rule, appId, dodPercent, rule.Threshold,
                        $"Daily revenue changed {dodPercent:+0.0;-0.0}%"));
                }
            }
        }

        return alerts;
    }

    private bool EvaluateCondition(decimal value, string op, decimal threshold)
    {
        return op switch
        {
            "<" => value < threshold,
            ">" => value > threshold,
            "<=" => value <= threshold,
            ">=" => value >= threshold,
            "=" => value == threshold,
            "!=" => value != threshold,
            _ => false
        };
```

```
    }

    private AlertHistory CreateAlert(AlertRule rule, string? appId, decimal metricValue, decim
al threshold, string message)
    {
        return new AlertHistory
        {
            RuleId = rule.Id,
            RuleCode = rule.Code,
            AppId = appId,
            TriggeredAt = DateTime.UtcNow,
            MetricValue = metricValue,
            ThresholdValue = threshold,
            Message = message,
            Status = "PENDING"
        };
    }

    public async Task<bool> ShouldDeduplicateAsync(int ruleId, string? appId, string? mgId)
    {
        var rule = await _ruleRepo.GetByIdAsync(ruleId);
        if (rule == null) return true;

        var recent = await _historyRepo.GetRecentAlertAsync(ruleId, appId, mgId, rule.Cooldo
wnHours);
        return recent != null;
    }
}
```

10.4 Hangfire Job

```
public class AlertCheckJob
{
    private readonly IAlertService _alertService;
    private readonly ILogger<AlertCheckJob> _logger;

    public AlertCheckJob(IAlertService alertService, ILogger<AlertCheckJob> logger)
    {
        _alertService = alertService;
        _logger = logger;
    }

    [AutomaticRetry(Attempts = 3)]
    public async Task ExecuteAsync()
    {
```

```
        _logger.LogInformation("Starting alert check job at {Time}", DateTime.UtcNow);

        try
        {
            var result = await _alertService.EvaluateAllRulesAsync();

            _logger.LogInformation(
                "Alert check completed. Triggered: {Triggered}, Skipped: {Skipped}, Errors: {Erro
rs}",
                result.TriggeredCount, result.SkippedCount, result.ErrorCount);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Alert check job failed");
            throw;
        }
    }
}

// Registration in Program.cs
services.AddHangfire(config => config.UsePostgreSqlStorage(connectionString));
services.AddHangfireServer();

// Schedule job
RecurringJob.AddOrUpdate<AlertCheckJob>(
    "alert-check",
    job => job.ExecuteAsync(),
    "*/15 * * * *"); // Every 15 minutes
```

SUMMARY

Quick Reference

Category	Rules	Priority Mix
REVENUE	5	3 HIGH, 2 MEDIUM
PERFORMANCE	5	2 HIGH, 3 MEDIUM
WATERFALL	5	0 HIGH, 3 MEDIUM, 2 LOW
SYSTEM	5	3 HIGH, 2 MEDIUM
RECOMMENDATION	4	0 HIGH, 1 MEDIUM, 3 LOW
TOTAL	**24**	**8 HIGH, 11 MEDIUM, 5 LOW**

Cooldown Summary

Priority	Cooldown	Channels
🔴 HIGH	4 hours	All (Telegram, Lark, Slack, Email)
🟡 MEDIUM	8 hours	Telegram, Lark
🟢 LOW	24 hours	Dashboard only

Implementation Checklist

- [ ] Database tables created
- [ ] Initial alert rules inserted
- [ ] Telegram bot configured
- [ ] Lark webhook configured
- [ ] AlertService implemented
- [ ] Hangfire job scheduled
- [ ] API endpoints created
- [ ] Dashboard UI for alert management
- [ ] Testing completed

Document Version: 1.0 | Last Updated: January 2025